INTERNATIONAL SESSION

ecoloop

circular material systems & zero-cost digital architecture

overview

a physical sustainability project combining verified upcycled materials with modern web infrastructure, built autonomously using agentic coding tools.

6 VERIFIED PRODUCTS | REACT 18 + TS | $0 CLOUD COSTS



[image for hero visual / platform preview]

clean logo, project render, or ecoloop preview

THE PROBLEM

the linear packaging trap

why conventional municipal recycling breaks down on complex packaging

material reality

LINEAR

- **multi-layer packaging (mlp)**
  chips and snack wrappers bond plastic and aluminum together. recyclers cannot separate them profitably, so they go directly to landfills.
- **zero supply tracking**
  schools and communities generate tons of clean packaging waste without verified channels to divert materials into local production loops.

the ecoloop objective

CIRCULAR

- **direct mechanical upcycling**
  heat-press multi-layer plastics into structural sheets that replace plywood and acoustic boards.
- **digital transparency**
  publish physical specifications, density metrics, and inventory through an open platform.

MATERIAL CATALOG

the physical loop

six verified circular materials diverted from waste streams

eco-bricks

high-density pet bottles packed with clean flexible plastics

mlp sheets

heat-pressed snack wrappers replacing construction plywood

bottle planters

upcycled containers for nursery and hydroponic gardening

uniform bags

heavyweight tote bags repurposed from discarded school uniforms

seed paper

zero-bleach handmade paper embedded with native seeds

bioenzymes

citrus-fermented multipurpose natural cleaning solutions



REALITY CHECK // 01

how it started vs how it's going



[meme: me explaining to teachers that collecting plastic wrappers requires a serverless cloud architecture]

clown applying makeup / roll safe / spongeBob presentation

started as collecting empty chips wrappers in school. ended up designing automated distributed cloud infrastructure.

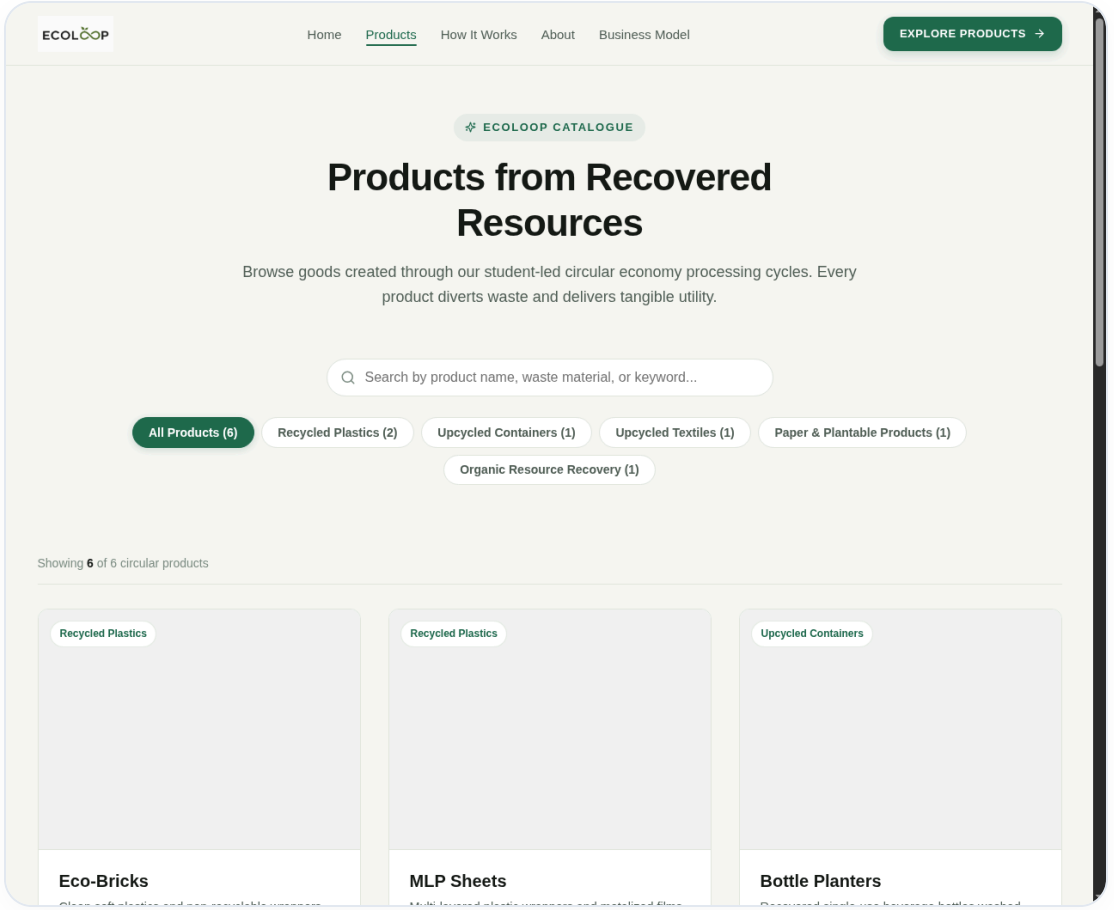
DIGITAL LAYER

why physical projects need software

moving from static school exhibits to an open, operational platform

platform capabilities

- **engineering specifications**
  every product includes density, dimensions, thermal ratings, and manufacturing lead times.
- **interactive transaction routing**
  partners can query inventory, request institutional samples, and place bulk orders seamlessly.
- **modern frontend stack**
  react 18 + typescript + vite for sub-50ms navigation and zero compile-time errors.



SERVERLESS INFRASTRUCTURE

the zero-cost backend

delivering full transactional workflows without servers or monthly bills

STEP 01

react client

user submits an inquiry; client validates types and dispatches json payload

→

STEP 02

apps script webhook

stateless cloud endpoint parses payload, handles cors, and formats data

→

STEP 03

sheets datastore

appends structured records, updates inventory counters, triggers email notifications

total infrastructure cost

no credit cards, no servers to maintain, 99.9% uptime

$0.00 / month

REALITY CHECK // 02

the cloud bill dilemma



[meme: buff doge vs cheems - $450/mo enterprise aws vs $0 google sheets backend]

buff doge vs cheems / who would win

enterprise developers arguing about kubernetes microservices vs two high schoolers using google sheets as a relational database.

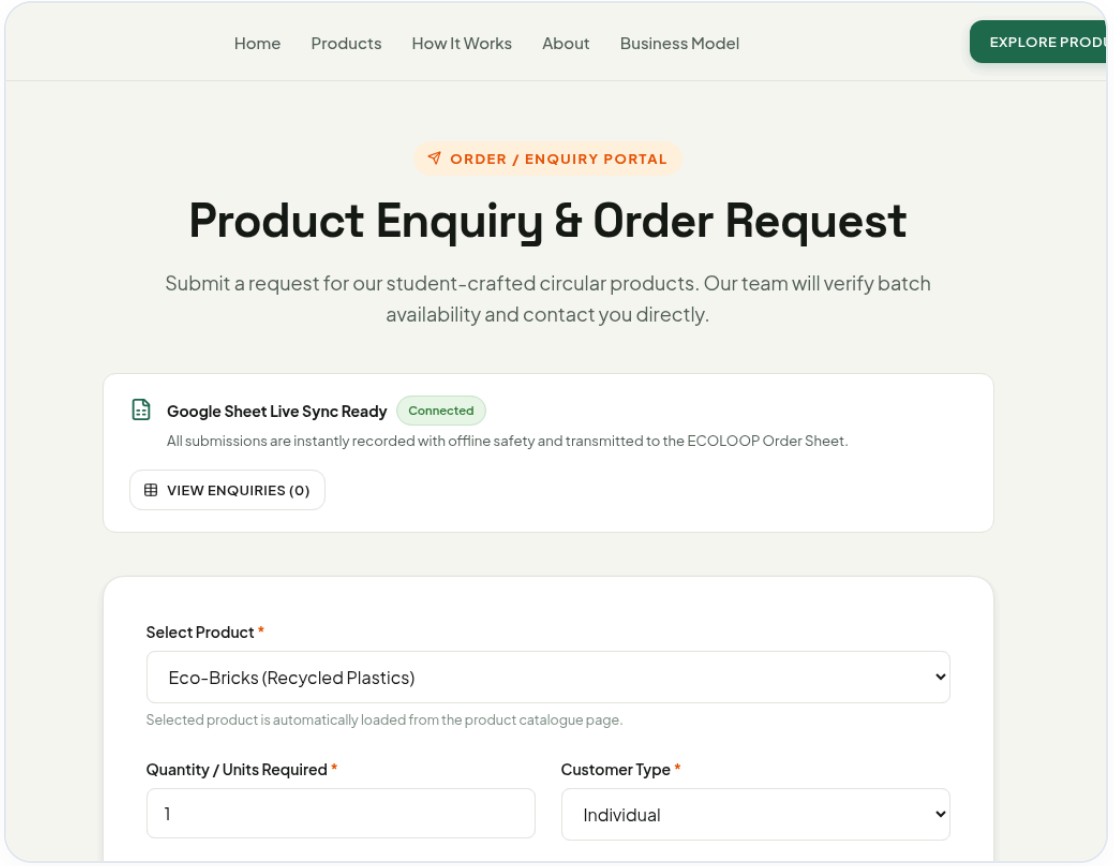
USER EXPERIENCE

interactive inquiry flow

handling inquiries and institutional orders with context preservation

order & sample lifecycle

- **contextual prefill**
  clicking inquire automatically binds product id, material specs, and default unit quantities into form state.
- **zero-friction validation**
  client-side schema validation flags incomplete entries before initiating network requests.
- **instant feedback**
  live confirmation modal informs users of lead times and logs inquiries to our central spreadsheet.



ENGINEERING METHODOLOGY

the shift: developer as architect

how agentic workflows replace manual syntax wrestling with system design

traditional coding

LEGACY

- manual typing of boilerplate and repetitious component state.
- spending 3 hours searching documentation to fix broken npm imports.
- debugging subtle syntax errors instead of focusing on user experience.

agentic coding

MODERN

- developer defines strict specifications, system constraints, and types.
- autonomous agents read repo structure, modify files, and run builds.
- the compiler validates changes; developer reviews diffs and commits.

REALITY CHECK // 03

documentation in 2026



[meme: drake hotline bling - reading 400 pages of framework documentation vs writing an AGENTS.md spec and letting the agent compile]

drake hotline / galaxy brain

reading 200 pages of api documentation at 3 am is officially obsolete. write clean constraints and let the tools execute.

THE ENGINE

the closed-feedback agentic loop

it is not copy-pasting answers; it is an automated engineering cycle

1. specification

RULES

define boundaries in `AGENTS.md`: design tokens, allowed libraries, performance budgets, and file structures.

2. autonomous edit

AGENT

the agent inspects the repository, plans file changes, and modifies code across multiple files simultaneously.

3. compiler verification

TRUTH

automated execution of `tsc` and `vite build`. if errors occur, the agent reads terminal output and repairs itself.

4. git checkpoint

PERSISTENCE

atomic git commits at each verified step guarantee a clean recovery path if an experiment fails.

OPEN SOURCE

introducing opencode

an open, extensible autonomous coding agent that runs directly in your terminal

why opencode?

- **100% open source**
  built by anomaly (github: `anomalyco/opencode`). zero vendor lock-in; connect to any model endpoint.
- **dual execution modes**
  `plan mode` surveys codebase structure without editing; `build mode` executes verified changes.
- **native developer toolchain**
  integrates with git, bash tools, local linters, and compilers out of the box.

// quick installation

```
$ npm install -g @opencode/cli
$ cd ./my-project
$ opencode
```

// works on any mac, linux, or windows machine

ACCESSIBILITY

the $0 student setup

how to build software with autonomous agents without spending a penny

tier 1: fast cloud (free)

CLOUD

- **groq api**
  blazing fast inference with generous daily free tiers. runs open models like llama 3.3.
- **openrouter free models**
  access to free community endpoints for prototyping and reasoning without a credit card.

tier 2: 100% offline (private)

LOCAL

- **ollama local runtime**
  run models entirely on your laptop hardware. zero internet connection required.
- **recommended model**
  `qwen2.5-coder:7b` offers state-of-the-art coding reasoning on standard consumer laptops.

REALITY CHECK // 04

the compiler never lies



"i fixed all the bugs" says the ai model.
typescript compiler: 47 errors found in 3 files.

CONCLUSION

ecoloop • open discussion

collaborative ideas, material exchanges, and live platform demo

core engineering takeaways

- **specifications matter:** agents are only as sharp as the constraints you give them.
- **compiler is ground truth:** always let real compilers and tests validate output.
- **commit constantly:** small, verified git commits make experimentation safe.

thank you.

explore the platform code, try the inquiry engine, or experiment with open-source agents.

`github.com/Harman209/ecoloop`